

Dron de superficie a vela

Informe técnico — Desarrollo del firmware y de la electrónica

Plataforma:	LilyGO T-Beam V1.1 (ESP32 + SX1276 + GPS u-blox +
Lenguaje:	Arduino / C++, destino ESP32 Arduino Core 3.x
Autor (código original):	Mohamed EL JILY
Autor (refonte / firmware actual):	Facundo Arito
Fecha:	June 13, 2026

Resumen

Este documento recorre la totalidad del desarrollo informático y electrónico del dron de superficie a vela basado en el módulo T-Beam de Espressif/LilyGO. Cubre el concepto físico del vehículo, el código original y sus limitaciones, la reescritura completa de la arquitectura de software, las dos revisiones del PCB, el modo manual unificado con propulsión bidireccional, la navegación autónoma a vela, la interfaz de estación terrena (IHM) y, de forma detallada, **todos los problemas** encontrados a lo largo del proyecto junto con las soluciones implementadas. Una sección final reúne los problemas aún abiertos y las opciones de mejora propuestas para resolverlos.

Proyecto de investigación (PER) — Dron autónomo de adquisición de datos marinos

Índice

1	Guía de uso para el próximo grupo de trabajo	2
1.1	Material necesario	2
1.2	Puesta en marcha del dron (barco)	2
1.3	Programación (flasheo) de las T-Beam	3
1.4	Conexión de las dos T-Beam	4
1.5	Uso de la IHM en Linux	4
1.5.1	Programas que hay que instalar (Linux)	4
1.6	Uso de la IHM en Windows	6
1.6.1	Programas que hay que instalar (Windows)	6
2	Presentación del proyecto	7
2.1	Concepto físico	7
2.2	Objetivo del proyecto	8
2.3	Material principal	8
3	Código original — <code>boat.ino</code>	9
3.1	Arquitectura y funcionamiento	9
3.2	Problemas y limitaciones identificados	9
3.2.1	Biblioteca ESP32Servo — conflicto LEDC / interrupciones . . .	9
3.2.2	Asignación de pines RC — conflicto I2C / AXP192	9
3.2.3	Pin de batería GPI035 — drenaje de 20 mA continuo	9
3.2.4	Pin LoRa RST — error de documentación	10
3.2.5	Gestión de ESC — sin armado ni limitación de variación	10
4	PCB Versión 1 — Placa de interfaz original	10
5	Reescritura del firmware — Arquitectura Post-Claude	10
5.1	Motivaciones	11
5.2	Arquitectura en capas	11
5.3	Decisiones técnicas fundamentales	12
5.3.1	MCPWM obligatorio — nunca LEDC	12
5.3.2	Interrupciones RC con protección portMUX	12
5.3.3	Todas las consignas de actuadores en microsegundos	13
5.3.4	Slew rate — protección contra tirones	13
6	Fase 1 — Control manual por radio	13
6.1	Realización	14

6.2	Modos de vuelo (selector CH5) — versión unificada actual	14
6.3	Modo manual unificado	14
6.4	Propulsor bidireccional (marcha adelante / atrás)	15
6.5	Resultados de las pruebas — Fase 1	15
7	Fase 2 — GPS y navegación	16
7.1	Problema crítico — GPS mudo tras configuración en flash	16
7.2	Problema crítico — antena activa sin alimentación	16
7.3	Nota sobre el arranque en frío	17
8	Fase 3 — LoRa y telemetría	17
8.1	Protocolo JSON — heartbeat actual	17
8.2	Comandos (tierra → dron)	18
9	PCB Versión 2 — Mejoras de hardware	18
9.1	Correspondencia de pines (PCB V2)	18
9.2	Amplificación de nivel lógico — transistores BC548	19
9.3	Medición de batería — divisor de alta impedancia	19
9.4	AXP192 — I2C persistente	20
10	Navegación autónoma (Fases 5 y 6)	20
10.1	Fase 5 — Estimación de viento desde el track GPS	20
10.2	Fase 6 — Algoritmo de navegación a vela	20
11	Interfaz de estación terrena (IHM)	21
12	Problemas resueltos y soluciones detalladas	21
12.1	Alimentación del T-Beam — imposible alimentar desde los pines	21
12.2	Motor que funciona “a impulsos” — corte por baja tensión	22
12.3	Fallo al programar el barco — brownout durante el flash	23
12.4	Otros problemas de firmware ya resueltos	23
12.4.1	DUTY_MODE_1 en OPR_B — servo del timón inoperante . .	23
12.4.2	Umbral de armado de ESC demasiado bajo	23
12.4.3	Modo bloqueado en Automatic — confusión CH4/CH5	23
12.4.4	Puerto serie ocupado al programar	23
13	Problemas que encontramos — opciones de mejora	24
13.1	Alcance de LoRa muy pobre dentro de la caja de electrónica	24
13.2	Inconsistencias del GPS	24
13.3	Sin medición de viento ni de la posición de la vela	25
13.4	Poco espacio para las baterías en la caja	25

14 Estado actual del sistema	26
14.1 Valores de calibración actuales (resumen)	26

1. Guía de uso para el próximo grupo de trabajo

Esta sección es una **guía práctica de puesta en marcha** pensada para el siguiente grupo que retome el proyecto. Explica qué material hace falta, cómo encender y pilotar el dron, cómo programar y conectar las dos placas T-Beam, y cómo levantar la interfaz de estación terrena (IHM) tanto en Linux como en Windows. Se recomienda leerla **completa** antes del primer encendido.

1.1. Material necesario

Elemento	Detalle
Dron (“barco”)	T-Beam montada en el casco sobre el PCB V2. Conversor USB-serie CP2104, número de serie 02126CF1
Transceptor (estación tierra)	Segunda T-Beam con el sketch <code>transceiver.ino</code> . CP2104 serie 01C00B54
Radio RC	Emisor Pro-Tronik PTR-6A + receptor R8X (2,4 GHz)
Batería	LiPo 2S cargada ($\approx 8,4$ V) + portabaterías del casco. Imprescindible para un funcionamiento estable
Computadora	Linux (recomendado) o Windows, con Docker, Python 3 y navegador
Cables USB	Uno de buena calidad para el transceptor; otro para programar / alimentar el barco
Tarjeta EPRG-3	Para programar el número de celdas y el corte por baja tensión (LVC) del ESC, y habilitar el modo bidireccional

1.2. Puesta en marcha del dron (barco)

1. **Cargar la LiPo** por completo y conectarla al **portabaterías** / **conector JST** del barco. **Nunca** alimentar el T-Beam inyectando 5 V por los pines del header (ver Sección 12.1); la fuente fiable es la LiPo.
2. **Encender primero el emisor RC** y comprobar que el selector de modo (CH5) está en **Manual** (posición alta). El **primer ensayo debe ser siempre manual**.

3. **Energizar el barco.** En el arranque imprime por serie líneas de estado (p. ej. [LORA] OK).
4. **Esperar el fix GPS:** el LED TIMEPULSE del NEO-6M parpadea a 1 Hz cuando hay fix. Con LiPo conectada el fix llega en segundos (arranque en caliente); sin LiPo, cada encendido es un arranque en frío de ≈ 2 min.
5. **Pilotar en manual:**
 - **CH5 (modo):** bajo = Automático, medio = Sail (inerte, todo en neutro), alto = Manual.
 - **CH2 (vela):** bascula la vela entre babor y estribor ($\pm 10^\circ$).
 - **CH4 (timón):** posiciona el winch del timón ($\pm 90^\circ$).
 - **CH3 (motor, bidireccional):** centro = paro, un extremo = +100 % adelante, el otro = -100 % atrás. Requiere el ESC en modo bidireccional (ver punto siguiente).
6. **Antes de usar el motor:** programar el ESC con la **tarjeta EPRG-3** (número de celdas, LVC y modo bidireccional). Si no, la marcha atrás no funciona y, con batería baja, el motor se corta “a impulsos”.

Seguridad: si se pierde la señal RC, el dron entra en *Failsafe*, que **navega de forma autónoma**. Como el modo autónomo nunca se validó en el agua, el primer ensayo debe hacerse en manual, en aguas tranquilas y con un plan de recuperación (p. ej. bote o cabo).

1.3. Programación (flasheo) de las T-Beam

Ambas placas usan el mismo núcleo (FQBN = `esp32:esp32:t-beam`). El barco lleva el sketch `main/`; el transceptor lleva `transceiver/`.

Listing 1: Compilar y flashear con arduino-cli

```
# (rutas relativas a la carpeta raíz del proyecto = Post-Claude)
# Compilar el firmware del barco
~/bin/arduino-cli compile --fqbn esp32:esp32:t-beam main

# Flashear el barco (ajustar el puerto)
~/bin/arduino-cli upload --port /dev/ttyUSB0 --fqbn esp32:esp32:t-beam main
```

```
# Idem para el transceptor
~/bin/arduino-cli upload --port /dev/ttyUSB0 --fqbn esp32:esp32:t-
beam transceiver
```

- **Identificar cada placa por su número de serie** (en Linux: `ls -l /dev/serial/by-id/`). Barco = 02126CF1, transceptor = 01C00B54.
- **Si el flasheo falla** (“*chip stopped responding*”, el puerto se re-enumera): es un *brownout* por alimentación insuficiente. Alimentar con LiPo cargada y/o entrar en modo bootloader manual (mantener **BOOT/IO0** mientras se pulsa **RST** o se reconecta el USB). Ver Sección 12.1.
- No tener un monitor serie abierto sobre el puerto mientras se programa.

1.4. Conexión de las dos T-Beam

- El **transceptor** se conecta por **USB a la computadora** de tierra. Aparece como `/dev/ttyUSBx` en Linux o `COMx` en Windows.
- El **barco no se conecta por cable** durante la operación: se comunica con el transceptor por **radio LoRa a 433 MHz**. Ambos firmwares ya usan los mismos parámetros de radio (433 MHz, SF7, BW125 kHz).
- El transceptor reenvía a la IHM cada heartbeat del barco (y le inyecta el RSSI medido en tierra) y transmite al barco los comandos de la IHM.

No abrir un monitor serie (gtkterm, el monitor del IDE Arduino, un script pyserial, etc.) sobre el puerto del transceptor **mientras corre la IHM**: los dos lectores compiten por el puerto y corrompen el JSON de telemetría.

1.5. Uso de la IHM en Linux

1.5.1. Programas que hay que instalar (Linux)

Programa	Para qué / cómo
Docker Engine + plugin docker compose	Levanta la base de datos MongoDB (imagen mongo:7, que se descarga sola la primera vez). En Ubuntu/Debian: <code>sudo apt install docker.io docker-compose-plugin</code> . Añadir el usuario al grupo: <code>sudo usermod -aG docker \$USER</code> (y reabrir sesión) para no usar sudo
Python 3 + venv + pip Paquetes Python	Ejecuta el servidor y el puente serie. <code>sudo apt install python3 python3-venv python3-pip</code> Se instalan dentro de un entorno virtual a partir de <code>requirements.txt</code> : fastapi, uvicorn, pymongo[asyncio], pyserial, python-dotenv
Navegador web	Para abrir la interfaz (Firefox, Chrome, etc.)

Preparación (una sola vez) — crear el entorno virtual e instalar los paquetes Python (rutas relativas a la raíz Post-Claude):

Listing 2: Instalacion inicial de la IHM en Linux

```
cd IHM
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
```

Arranque y parada — conectar el transceptor por USB y, desde la carpeta IHM/, ejecutar:

Listing 3: Arrancar y detener la IHM en Linux

```
cd IHM
./start_ihm.sh                # autodetecta el transceptor por su
                             serie
# o forzar el puerto:
./start_ihm.sh /dev/ttyUSB0

# ... al terminar:
./stop_ihm.sh                 # libera el puerto 5000 y baja MongoDB
                             (Docker)
```

- `start_ihm.sh` levanta MongoDB (contenedor Docker via `docker compose`), `serial_link.py` (lee/escribe el puerto serie) y `webserver.py` (servidor web), y abre el navegador en `http://localhost:5000`.

- En el mapa: ver telemetría en vivo, definir waypoints con clic, enviar la ruta y los comandos (navegar, parar, medir viento, etc.).
- Conmutador **Sim** / **Réal**: en “Réal” usa el transceptor físico; en “Sim” levanta un simulador de barco (sin hardware).
- El puerto del transceptor se **autodetecta** por su número de serie (01C00B54), por lo que normalmente no hace falta indicarlo.

1.6. Uso de la IHM en Windows

La IHM se desarrolló en Linux (los scripts de arranque son **bash**), por lo que en Windows lo más sencillo es ejecutarla dentro de WSL2.

1.6.1. Programas que hay que instalar (Windows)

Programa	Para qué / cómo
WSL2 (Ubuntu)	Subsistema Linux dentro de Windows; permite usar los mismos scripts bash . Instalar con wsl -install en PowerShell (como administrador)
Docker Desktop	Levanta MongoDB. Instalarlo y activar la integración con WSL2 en sus ajustes
usbipd-win	Comparte el puerto USB del transceptor con WSL (Windows no lo pasa solo). Instalar con winget install usbipd
Python 3 + paquetes	Ya viene en Ubuntu de WSL; instalar los paquetes igual que en Linux (requirements.txt). Si se usa Windows nativo, instalar Python desde python.org
Navegador web	Para abrir http://localhost:5000

Camino recomendado — WSL2 + Docker Desktop: dentro de Ubuntu (WSL) se siguen **exactamente los mismos pasos que en Linux** (crear el **.venv**, instalar **requirements.txt**, **./start_ihm.sh**). Para que WSL “vea” el transceptor hay que adjuntar el USB desde PowerShell:

Listing 4: Adjuntar el USB del transceptor a WSL (PowerShell admin)

```
usbipd list                                # ver el BUSID del CP210x (
    transceptor)
usbipd bind --busid <BUSID>
```

```
usbipd attach --wsl --busid <BUSID>    # dentro de WSL aparece como /  
dev/ttyUSBx
```

Alternativa — Windows nativo (sin WSL): instalar Docker Desktop y Python para Windows y lanzar los componentes a mano: `docker compose up` para MongoDB, luego `python serial_link.py` y `python webserver.py`. El puerto serie será COMx (verificarlo en el Administrador de dispositivos) y hay que ajustar `SERIAL_PORT_REAL` en el archivo `.env` a ese COMx. En ambos casos la interfaz se abre en `http://localhost:5000`.

Red WiFi — EDUROAM no funciona. Como la IHM usa MongoDB (en Docker) y la pila de red local, en la red institucional **EDUROAM la IHM no funciona**: EDUROAM aplica aislamiento de clientes y restricciones de red (firewall/proxy) que impiden el correcto funcionamiento de Docker/MongoDB y de la comunicación local necesaria.

Solución (probada por el grupo): conectar la computadora que ejecuta la IHM a una red sin esas restricciones. En la práctica, **compartir los datos móviles del celular (hotspot / anclaje a red) a la computadora**: con el hotspot del teléfono la IHM levanta y funciona correctamente.

2. Presentación del proyecto

2.1. Concepto físico

El dron es un vehículo de superficie propulsado principalmente por una vela. Su concepción es original: la vela es de **perfil de ala** y gira libremente alrededor de un eje vertical solidario al casco. Su aerodinámica pasiva orienta automáticamente el perfil de modo que el viento incide siempre a unos 45° del eje longitudinal del dron, sin necesidad de un control activo del ángulo de la vela. La maniobra se realiza con dos actuadores principales:

- **Servo alerón (vela) — Futaba S3003:** controla un pequeño flap en el borde de fuga de la vela. Su recorrido útil es estrictamente de $\pm 10^\circ$. La deflexión determina hacia qué lado (babor o estribor) empuja la vela, eligiendo de hecho la amura sobre la que navega el barco. **No es un control continuo**: solo las dos posiciones extremas tienen sentido físico, por lo que se modela como un estado binario (+1 / -1).

- **Servo timón / rotor — Graupner Regatta ECO II (mod. 5176):** es el actuador principal de rumbo (guiniada). Es un *winch de vela multivuelta posicional*: la señal PWM ordena una **posición** del eje (no una velocidad). Centro mecánico = 1500 μ s, recorrido total = 6 vueltas, y mantiene la posición bajo carga como un servo estándar.
- **Propulsor (ESC):** utilizado únicamente cuando no hay viento suficiente. Permite la propulsión a hélice como respaldo. En el PCB V2 se usa un único ESC (se eliminó el segundo).

El sistema **no mide** la dirección del viento directamente: la *deduce* a partir del rumbo GPS (track) y del estado de los actuadores. La estimación requiere un desplazamiento mínimo de 30 m. Esta carencia de sensores es central en el proyecto y se retoma en la Sección 13.

2.2. Objetivo del proyecto

El proyecto es un PER (Proyecto de Estudios y Recherche) cuyo propósito final es la **adquisición autónoma de datos marinos**: el dron debe alcanzar puntos de paso (*waypoints*) GPS navegando a vela, recolectar datos de sensores y transmitirlos por radio LoRa a una estación terrena.

2.3. Material principal

Componente	Descripción
LilyGO T-Beam V1.1	Módulo que integra ESP32 WROVER, LoRa SX1276 433 MHz, GPS u-blox NEO-6M, AXP192 (gestión de energía) y portabaterías 18650
Futaba S3003	Servo alerón de la vela, centro a 1520 μ s, $\pm 10^\circ$
Graupner Regatta ECO II	Winch de timón multivuelta, 1500 μ s = centro, 1000–2000 μ s = rango, hasta 3,3 A en carga
Pro-Tronik PTR-6A	Emisor 6 vías FHSS 2,4 GHz, PWM 1000–2000 μ s
Pro-Tronik R8X	Receptor 8 vías, PWM estándar 50 Hz
Pro-Tronik Black Fet	ESC de la serie, programación por tarjeta EPRG-3
Antena GPS	Taoglas activa, conector u.FL
PCB v1 / PCB v2	Placa de interfaz a medida (KiCad 9)

3. Código original — `boat.ino`

3.1. Arquitectura y funcionamiento

El código original (`boat.ino`, del primer grupo, fuera de este repositorio), escrito por Mohamed EL JILY, constituye el punto de partida del proyecto. Estaba organizado en módulos: `Gps`, `ServoControl`, `RadioReceiver`, `MotorControl` y la lógica de navegación en `boat.ino`. Incluía ya una lógica de navegación a vela (ángulo relativo viento/rumbo, cambio de amura automático, orzar/arribar) y recepción de waypoints por LoRa en JSON.

3.2. Problemas y limitaciones identificados

Al retomar el proyecto se identificaron varios problemas bloqueantes que, en conjunto con la necesidad de nuevas features discutidas con el cliente y el profesor, motivaron la reescritura completa del firmware.

3.2.1. Biblioteca ESP32Servo — conflicto LEDC / interrupciones

La biblioteca `ESP32Servo` utiliza el periférico **LEDC** del ESP32. LEDC es **incompatible** con la decodificación de PWM por interrupciones GPIO usada para leer la radio: cuando LEDC está activo, las interrupciones del receptor RC se perturban y los canales leídos **vuelven constantemente a cero**, aunque las señales de entrada sean correctas en el osciloscopio. Esto hace imposible controlar los servos y leer la radio al mismo tiempo. Fue el problema más crítico del proyecto.

3.2.2. Asignación de pines RC — conflicto I2C / AXP192

El código original asignaba canales RC a `GPIO21` y `GPIO22`, que son los pines I2C internos del T-Beam usados por el AXP192 (gestor de energía). Además, `GPIO23` está conectado internamente al RESET del SX1276 (LoRa): conectar una interrupción en ese pin podía provocar resets espúreos de la radio.

3.2.3. Pin de batería `GPIO35` — drenaje de 20 mA continuo

El T-Beam V1.1 tiene un divisor resistivo de *baja impedancia* ($\approx 370\ \Omega$ total) soldado de forma permanente en `GPIO35`. Aun sin leer el ADC, ese divisor consume $\approx 20\text{ mA}$

continuos ($\approx 1,8$ W) directamente de la batería. **Nunca debe leerse GPI035.**

3.2.4. Pin LoRa RST — error de documentación

La documentación de LilyGO y muchos ejemplos indican GPI014 para el RESET del SX1276. El hardware del T-Beam V1.1 conecta físicamente el RESET en GPI023.

3.2.5. Gestión de ESC — sin armado ni limitación de variación

El código original no implementaba ninguna función del ESC dado que fue una feature a futuro, al igual que la medición de batería.

4. PCB Versión 1 — Placa de interfaz original

La primera placa realizaba la interfaz entre el T-Beam y los actuadores (conectores de 3 pines para servos/ESC, conectores RC, regulador reductor 5 V Pololu, bornes de potencia). Sus problemas se resumen a continuación.

Problema	Impacto
PWM 3,3 V sin amplificación de nivel	Servos y ESC (lógica 5 V) pueden no detectar correctamente los pulsos
Pines RC en GPI021/GPI022	Conflicto I2C/AXP192
ESC en GPI032/GPI033	Coinciden con LoRa DIO1/DIO2 del T-Beam
GPI035 para batería	Divisor de $370\ \Omega = 20$ mA continuos
Sin protección de polaridad	Riesgo ante alimentación incorrecta

5. Reescritura del firmware — Arquitectura Post-Claude

5.1. Motivaciones

La reescritura completa busca corregir todas las limitaciones identificadas y preparar la arquitectura para las fases autónomas. Objetivos:

1. Usar **MCPWM** en lugar de LEDC para las salidas de actuadores.
2. Decodificación RC **enteramente por interrupciones** con protección ISR.
3. Arquitectura orientada a objetos en capas, extensible por fases.
4. Armado de ESC y limitación de variación (*slew rate*).
5. Gestión limpia de la energía vía AXP192.
6. GPS, LoRa y batería funcionando simultáneamente.

5.2. Arquitectura en capas

Listing 5: Árbol del proyecto (raiz = Post-Claude)

```

Post-Claude/                <- carpeta raiz del proyecto
  main/main.ino              <- Punto de entrada (begin / update)
  main/src/
    app/DroneApp.cpp          <- Orquestador (scheduler por software
      )
    config/BoardConfig.h      <- Asignacion de GPIO y umbrales RC
    config/Calibration.h      <- Valores en microsegundos (servo/ESC
      )
    core/Types.h              <- Tipos compartidos (RcFrame,
      ActuatorCommand)
    control/ModeManager        <- CH5 -> modo de control
    control/ManualController   <- Modo manual unificado (vela+timon+
      motor)
    control/AutoController     <- Envoltura de la navegacion autonoma
    drivers/RcReceiver          <- Lectura PWM por interrupciones
    drivers/McpwmActuators      <- Salidas MCPWM (vela, timon, ESC)
    drivers/AxpPower            <- Init AXP192 (rieles GPS/LoRa/3.3V)
    drivers/BatteryAdc          <- ADC de tension de bateria
    drivers/GpsUart             <- Lectura GPS (TinyGPSPlus + UBX)
    drivers/LoRaRadio           <- Driver SX1276
    comm/LoRaComm              <- Protocolo JSON LoRa (heartbeat +
      comandos)
    navigation/                <- navigation.h, Navigator,
      MissionManager...
  transceiver/transceiver.ino  <- Estacion terrena (puente USB <->
    LoRa)

```

```
IHM/                                     <- Interfaz web de estacion terrena (
FastAPI)
```

El reparto sigue una lógica clara: *drivers* (hardware), *servicios / comm* (datos), *navigation* (geometría y misión), *control* (decisiones) y *app* (orquestración). El planificador es cooperativo (sin RTOS): `DroneApp::update()` dispara cada tarea cuando su período vence, evitando `delay()` que bloquearía la lectura RF.

5.3. Decisiones técnicas fundamentales

5.3.1. MCPWM obligatorio — nunca LEDC

Listing 6: `McpwmActuators.cpp` – inicializacion MCPWM

```
1 bool McpwmActuators::begin() {
2     mcpwm_gpio_init(MCPWM_UNIT_0, MCPWMOA,
3         BoardConfig::SAIL_SERVO_PIN);
4     mcpwm_gpio_init(MCPWM_UNIT_0, MCPWMOB,
5         BoardConfig::ROTOR_SERVO_PIN);
6     mcpwm_gpio_init(MCPWM_UNIT_0, MCPWM1A, BoardConfig::ESC1_PIN);
7     mcpwm_config_t cfg = {};
8     cfg.frequency      = Calibration::PWM_FREQ_HZ;    // 50 Hz
9     cfg.duty_mode      = MCPWM_DUTY_MODE_0;          // activo alto
10    cfg.counter_mode    = MCPWM_UP_COUNTER;
11    mcpwm_init(MCPWM_UNIT_0, MCPWM_TIMER_0, &cfg);    // vela + timon
12    mcpwm_init(MCPWM_UNIT_0, MCPWM_TIMER_1, &cfg);    // ESC
13    ...
14 }
```

El periférico MCPWM es independiente de las interrupciones GPIO: ambos coexisten sin perturbarse. Esta restricción es **innegociable** en este proyecto.

5.3.2. Interrupciones RC con protección portMUX

Listing 7: `RcReceiver.cpp` – ISR y seccion critica

```
1 void IRAM_ATTR RcReceiver::handleEdge(uint8_t pin, Ch ch) {
2     const uint32_t now = micros();
3     const int      idx = static_cast<int>(ch);
4     if (gpio_get_level(static_cast<gpio_num_t>(pin))) {
5         portENTER_CRITICAL_ISR(&mux_);
6         riseUs_[idx] = now;                                // flanco de
7         subida
8         portEXIT_CRITICAL_ISR(&mux_);
9     } else {
10        uint32_t rise;
11        portENTER_CRITICAL_ISR(&mux_);
```

```

11     rise = riseUs_[idx];
12     portEXIT_CRITICAL_ISR(&mux_);
13     const uint32_t width = now - rise;           // ancho de pulso
14     if (width >= RC_VALID_MIN_US && width <= RC_VALID_MAX_US) {
15         portENTER_CRITICAL_ISR(&mux_);
16         pulseUs_[idx] = static_cast<uint16_t>(width);
17         lastValidUs_[idx] = now;
18         portEXIT_CRITICAL_ISR(&mux_);
19     }
20 }
21 }

```

Las ISR se ubican en RAM (IRAM_ATTR) y se sincronizan con la boucle principal mediante portMUX (mecanismo FreeRTOS apropiado para el ESP32 dual-core). Un *timeout* de 100 ms marca un canal como perdido.

5.3.3. Todas las consignas de actuadores en microsegundos

Todos los actuadores se comandan exclusivamente en microsegundos, nunca en grados. Esto elimina una capa de conversión propensa a errores y corresponde directamente al protocolo PWM de servos y ESC.

5.3.4. Slew rate — protección contra tirones

Listing 8: McpwmActuators.cpp – limitacion de variacion

```

1  uint16_t McpwmActuators::slew(uint16_t current, uint16_t target,
   uint16_t step) {
2      if (current < target) {
3          uint32_t next = (uint32_t)current + step;
4          return (next >= target) ? target : (uint16_t)next;
5      }
6      if (current > target)
7          return (current > target + step) ? (uint16_t)(current -
           step) : target;
8      return current;
9  }
10 // ESC_SLEW_US = 30 us/tick, llamado cada 20 ms -> max 1500 us/s

```

6. Fase 1 — Control manual por radio

6.1. Realización

La Fase 1 cubre el control manual completo. Se implementaron los drivers (`AxpPower`, `RcReceiver`, `McpwmActuators`), el `ModeManager`, el `ManualController` y el orquestador `DroneApp` (ticks de 20 ms para control, 200 ms para debug, 1 s para LoRa).

6.2. Modos de vuelo (selector CH5) — versión unificada actual

El selector de 3 posiciones está cableado en CH5. La distribución de modos fue **reorganizada** respecto a la versión inicial (que tenía dos modos manuales separados “ManualServo” y “ManualProp”):

CH5	Modo	Comportamiento
$\leq 1250 \mu\text{s}$	Automatic	Navegación autónoma (sigue la misión LoRa)
1400–1600	Sail	Inerte : todas las salidas en neutro
$> 1800 \mu\text{s}$	Manual	Modo manual unificado (ver abajo)
Señal perdida	Failsafe	Sigue la misión autónoma (como Automatic)

Atención de seguridad: en pérdida de RC el sistema entra en *Failsafe*, que se comporta como el modo autónomo (navega solo). Como el modo autónomo nunca se probó en el agua, el **primer ensayo real debe ser manual** y con plan de recuperación.

6.3. Modo manual unificado

Un único modo (`computeManual`) controla a la vez la vela, el timón y el propulsor:

- **CH2 → vela (binaria):** banda muerta $\pm 35 \mu\text{s}$ alrededor de $1500 \mu\text{s}$. El primer frame inicializa el estado desde la posición del manche para evitar un salto al entrar en el modo. Dentro de la banda muerta se mantiene el último estado.
- **CH4 → timón (posicional):** el recorrido medido CH4 (1180–1790 μs) se mapea al rango físico del winch ($\text{ROTOR_MIN/MAX_US} = 1417\text{--}1583 \mu\text{s}$, $\pm 90^\circ$). Banda muerta central que fija el centro exacto (1500 μs).
- **CH3 → propulsor (bidireccional, ver Sección 6.4).**

Listing 9: `ManualController` – logica de vela binaria con estado memorizado

```
1 if (!sailStateInitialized_) {
```

```

2   sailState_ = (frame.ch2 >= RC_MID_US) ? +1 : -1;
3   sailStateInitialized_ = true;
4 }
5 if (frame.ch2 > RC_MID_US + db)      sailState_ = +1;
6 else if (frame.ch2 < RC_MID_US - db) sailState_ = -1;
7 cmd.sailUs = (sailState_ > 0) ? SAIL_PLUS_US : SAIL_MINUS_US;

```

6.4. Propulsor bidireccional (marcha adelante / atrás)

En la versión inicial el motor solo iba de 0 a 100 % (control inverso, con el ESC en modo unidireccional, 1000 μ s = paro). Se modificó para que el motor vaya de +100 % (adelante) a -100 % (atrás) usando el ESC en modo **bidireccional**, en el que el neutro pasa a 1500 μ s:

Posición del manche CH3	Potencia	ESC
CH3_FULL_US (1100 μ s)	+100 % adelante	2000 μ s
CH3_CENTER_US (1545 μ s) $\pm$ banda muerta	0 % paro	1500 μ s
CH3_ZERO_US (1990 μ s)	-100 % atrás	1000 μ s

Como el manche de gas es de tipo *trinquete* (no se autocentra), el paro se ubica en el centro de su recorrido con una banda muerta (ESC_CENTER_DEADBAND_US = 40 μ s). Para mantener coherente todo el firmware, el neutro 1500 μ s se usa también en el arranque, en Failsafe, en el modo Sail y como paro del modo autónomo; el límite inferior de saturación se bajó a 1000 μ s para permitir la marcha atrás.

La marcha atrás solo funciona físicamente si el ESC está **programado en modo bidireccional** con la tarjeta EPRG-3. Si no, todo valor por debajo de 1500 μ s se ignora y solo se obtiene marcha adelante.

6.5. Resultados de las pruebas — Fase 1

- [OK] Alerón conmuta correctamente entre 1465 μ s y 1575 μ s.
- [OK] Timón responde proporcionalmente al manche CH4.
- [OK] Failsafe se dispara en menos de 100 ms tras la pérdida de señal.

- [OK] Compilación `arduino-cli`: ≈ 28 % flash, 7 % RAM.
- [] Calibración de banco de `SAIL_PLUS/MINUS_US` y de `CH3_CENTER_US` pendiente.

7. Fase 2 — GPS y navegación

Se implementaron `GpsUart` (Serial1 en GPIO34/GPIO12, TinyGPSPlus), `Navigator` (haversine: distancia y rumbo), `MissionPlan` (hasta 16 waypoints) y `MissionManager` (máquina de estados Idle→Running→Returning→Complete). Dos problemas críticos de GPS se resolvieron en esta fase.

7.1. Problema crítico — GPS mudo tras configuración en flash

Síntoma: el contador `chars` se queda bloqueado, el GPS está alimentado pero no produce ninguna frase NMEA.

Causa: una sesión previa (IDE Arduino o u-center) guardó en la flash interna del NEO-6M una configuración CFG-PRT que desactiva la salida NMEA. Esa configuración sobrevive a los cortes de alimentación.

Enviar un reset de fábrica `UBX CFG-CFG` al arrancar (borra la configuración de flash y carga los valores de ROM). Implementado en `GpsUart::begin()`.

7.2. Problema crítico — antena activa sin alimentación

Síntoma: NMEA recibido pero `visible = 0` satélites incluso en exterior despejado y con antena externa conectada.

Causa: el supervisor de antena del NEO-6M está desactivado por defecto. Sin activarlo, no se entrega tensión de polarización al LNA de la antena activa y el conmutador RF del T-Beam permanece en la antena cerámica interna, cortocircuitando el conector u.FL.

Enviar `UBX CFG-ANT` con `flags = 0x001B` (control de tensión RF, detección de cortocircuito, recuperación). Tras la corrección: fix en menos de 60 s, 7 satélites,

HDOP $\approx 2,2$.

7.3. Nota sobre el arranque en frío

Sin batería LiPo conectada, cada encendido pierde el almanaque GPS: el arranque en frío tarda ≈ 2 min en exterior. Con batería se conservan las efémerides y el fix se obtiene en segundos (arranque en caliente).

8. Fase 3 — LoRa y telemetría

Se implementó el driver `LoRaRadio` (SX1276, SPI HSPI, 433 MHz), la capa de protocolo `LoRaComm` (heartbeat JSON a 1 Hz + despachador de comandos) y el sketch de estación terrena `transceiver.ino` (puente USB $\leftrightarrow$ LoRa con CLI compacta). Todo verificado en hardware (RSSI -101 a -106 dBm a corta distancia).

8.1. Protocolo JSON — heartbeat actual

El formato se afinó para mantenerse holgadamente por debajo del límite de 255 bytes de la FIFO del SX1276 (se eliminaron campos redundantes, se añadieron indicadores de salud GPS/RC, y el RSSI lo inyecta el transceptor en tierra):

Listing 10: Heartbeat dron -> tierra (1 Hz)

```
{"origin":"boat","type":"info","message":{
  "mode":"standby|route-ready|navigate",
  "location":[48.36000,-4.56000],
  "servos":{"sail":10,"rudder":-12},
  "heading":270,"wind":180,"bat":7.42,
  "fix":1,"sat":7,"hdop":2.2,"rc":1,
  "wt":3,"wc":1,"wobs":42,"rssi":-104}}
```

- `servos.sail` = $\pm 10^\circ$ (binaria); `servos.rudder` en grados del winch. `fix/sat/hdop` = salud GPS; `rc`=1 si el receptor entrega pulsos.
- `wt/wc` = waypoints total/actual. `wobs` = progreso de la medición de viento (0–100 %), presente *solo* mientras se mide.
- `rssi` **no** lo envía el barco: lo inyecta el transceptor en tierra, por lo que no cuenta en el presupuesto de 255 bytes.

8.2. Comandos (tierra → dron)

Listing 11: Comandos soportados

```
{
  "origin": "server",
  "type": "command",
  "message": {
    "navigate": true
  }
}
{
  "origin": "server",
  "type": "command",
  "message": {
    "stop": true
  }
}
{
  "origin": "server",
  "type": "command",
  "message": {
    "restart": true
  }
}
{
  "origin": "server",
  "type": "command",
  "message": {
    "wind-observation": true
  }
}
{
  "origin": "server",
  "type": "command",
  "message": {
    "wind-command": {
      "value": 225
    }
  }
}
{
  "origin": "server",
  "type": "command",
  "message": {
    "home": {
      "lat": 48.36,
      "lon": -4.56
    }
  }
}
{
  "origin": "server",
  "type": "command",
  "message": {
    "waypoints": {
      "number": 2,
      "points": "48.36,-4.56,48.37,-4.55"
    }
  }
}
```

LoRa.endPacket() es síncrono. A SF7/BW125 kHz, un paquete de ≈ 210 bytes tarda ≈ 450 ms, bloqueando la boucle principal una vez por segundo. Aceptable por ahora; conviene pasar a TX asíncrono antes de la navegación autónoma en agua si el jitter del lazo de control se vuelve crítico.

9. PCB Versión 2 — Mejoras de hardware

La V2 del PCB corrige todos los problemas de la V1 e incorpora los aprendizajes del desarrollo de software.

9.1. Correspondencia de pines (PCB V2)

Señal	GPIO	Notas
RC CH2 (alerón)	39	SVN, solo entrada
RC CH3 (gas)	14	Liberado (ya no comparte I2C)
RC CH4 (timón)	13	Liberado
RC CH5 (modo)	4	Conmutador 3 posiciones
Servo vela (PWM1)	2	Vía transistor BC548

Señal	GPIO	Notas
Servo timón (PWM2)	25	Vía transistor BC548
ESC1	15	Vía transistor BC548 (único ESC)
ADC batería	36	SVP, ADC1_CH0, solo entrada
LoRa	5/19/27/18	SPI HSPI
SCK/MISO/- MOSI/CS		
LoRa RST	23	Ahora libre (CH6 eliminado)
LoRa IRQ (DIO0)	26	
GPS RX / TX	34 / 12	Serial1
I2C SDA / SCL	21 / 22	AXP192 interno + conector de expansión

9.2. Amplificación de nivel lógico — transistores BC548

Las salidas PWM del ESP32 son de 3,3 V; los servos esperan 5 V. La V2 usa transistores NPN BC548 en emisor común.

Inicialmente el canal del timón (OPR_B) usaba `MCPWM_DUTY_MODE_1` (inversión), suponiendo que el NPN invertía la señal y había que preinvertir. En realidad el montaje BC548 ya invierte; aplicar `DUTY_MODE_1` producía una **doble inversión** que dejaba el servo del timón inoperante. La solución es usar `DUTY_MODE_0` (activo alto) en **ambos** canales.

9.3. Medición de batería — divisor de alta impedancia

Se usa un divisor $R5 = 562\text{ k}\Omega$ / $R6 = 120\text{ k}\Omega$ (razón 5,683) en `GPIO36` con atenuación de 11 dB. A 7,4 V el divisor consume $\approx 10,9\text{ }\mu\text{A}$, despreciable.

Detalle de montaje: en una placa las resistencias se soldaron invertidas (120 k arriba, 562 k abajo), dando una lectura de 17,9 V en lugar de $\approx 5\text{ V}$. El código era correcto; la solución fue invertir físicamente las resistencias.

9.4. AXP192 — I2C persistente

En la V1 se llamaba `Wire.end()` tras inicializar el AXP192 para liberar `GPIO21/GPIO22`. En la V2 esos pines quedan libres para I2C, por lo que `Wire.end()` se elimina: el bus I2C permanece activo y disponible para futuros sensores en el conector de expansión.

10. Navegación autónoma (Fases 5 y 6)

10.1. Fase 5 — Estimación de viento desde el track GPS

Sin sensor de viento, la dirección se infiere navegando sobre una amura fija: el barco fija la vela en $+10^\circ$, suaviza el rumbo GPS con una media móvil circular y, tras recorrer `WIND_OBS_DISTANCE_M = 30` m, fija el viento en `rumbo_suavizado +90°`. También existe un comando LoRa `wind-command` para forzar la dirección manualmente (respaldo fiable).

El desfase de $+90^\circ$ y el umbral de 30 m necesitan validación en campo. Además, durante la medición el heartbeat se ve igual que en reposo salvo por el campo `wobs`.

10.2. Fase 6 — Algoritmo de navegación a vela

El algoritmo de navegación (`navigation.h`) fue desarrollado por un compañero del grupo (rama `Testautoboat` del repositorio `DeltaGod/PER_MEA_LYINCROYAP`) e integrado en este firmware. Implementa:

- Zonas prohibidas de ceñida y empopada (*upwind/downwind*) con zigzag de borde.
- Evitación de trasluchada (*empannage*) mediante una máquina de estados.
- Corredor de tolerancia lateral (*cross-track*) alrededor de la línea al waypoint.
- Orzar/arribar (*lofer/abattre*) para el seguimiento fino de rumbo.

La última versión integrada aporta: selección de amura inicial más inteligente (`nav_sideMovingTowardW` que elige el lado que más progresa hacia el waypoint), detección de llegada por “plano del waypoint superado dentro del corredor” (`nav_hasPassedWaypoint`) y un corredor por defecto ampliado de 20 a 100 m. La envoltura `AutoController` mapea el rumbo de timón del algoritmo 1:1 a grados físicos del winch, limitado a $\pm 20^\circ$ (`ROTOR_AUTO_RANGE_DEG`), ya que el $\pm 90^\circ$ completo resultaba demasiado agresivo.

La navegación autónoma está **codificada pero nunca probada en el agua** y no tiene pruebas unitarias. Falta también un *WinchTracker*: el Regatta ECO II no tiene realimentación de posición, por lo que la posición real del timón no se conoce (ver Sección 13).

11. Interfaz de estación terrena (IHM)

La estación terrena consta de un segundo T-Beam con `transceiver.ino` (puente USB↔LoRa) y una aplicación web (IHM/) en Python FastAPI + MongoDB + un mapa Leaflet. Funciones principales:

- Mapa con posición del barco, waypoints (clic para definirlos), envío de ruta y *polling* de telemetría a 1 Hz.
- Panel de salud con semáforos (Enlace, GPS, Batería, RC, Control) y RSSI.
- Brújula de viento y predicción de trayectoria (réplica en JavaScript de `navigation.h`).
- Barra de progreso de la medición de viento.
- Autodetección del puerto del transceptor por número de serie del CP2104 (transceptor = 01C00B54, barco = 02126CF1).

12. Problemas resueltos y soluciones detalladas

Esta sección reúne, de forma exhaustiva, los problemas significativos encontrados durante el desarrollo y cómo se resolvieron.

12.1. Alimentación del T-Beam — imposible alimentar desde los pines

Problema: el T-Beam V1.1 **no se puede alimentar inyectando 5 V por los pines del header de expansión**. Las únicas vías de alimentación válidas son el portabaterías 18650/LiPo (conector JST de la propia placa) y el puerto USB. El PCB V2, en cambio, fue diseñado para alimentar el T-Beam a través del pin

+5V del header de expansión (salida del regulador reductor Pololu).

El motivo es la topología interna de energía del T-Beam: el pin +5V del header está situado en el camino del VBUS de USB, después del diodo de protección, y **no entra de forma fiable al AXP192** (el PMIC que habilita los rieles de 3,3 V, GPS y LoRa). En consecuencia, alimentando solo por ese pin el AXP192 a menudo **no arranca**, y bajo carga la tensión se desploma (se observó el barco corriendo a $\approx 0,81$ V, claramente en *brownout*), lo que además provoca la re-enumeración continua del conversor USB-serie (mensajes de kernel `cp210x ... status -19`).

Solución rápida adoptada: se cortó un cable USB-A $\rightarrow$ USB-C; el extremo USB-A (líneas +5V y GND) se **soldó directamente al riel de 5 V del PCB** (salida del Pololu), y el extremo USB-C se conecta al puerto USB del T-Beam. Así, la energía del PCB entra al T-Beam por su **camino de alimentación USB válido** (VBUS $\rightarrow$ AXP192), en lugar de por el pin +5V del header, que no era capaz de arrancar el PMIC.

Recomendación: esta solución es un *parche* funcional pero poco elegante (un cable USB soldado al riel). Una corrección definitiva en una futura revisión del PCB sería rutear la alimentación del PCB directamente a la entrada correcta del T-Beam (VBUS o el JST de batería), evitando el pin +5V del header.

12.2. Motor que funciona “a impulsos” — corte por baja tensión

Síntoma: con la batería a ≈ 7 V (según telemetría) el motor no gira de forma continua: arranca un instante, se corta, y solo vuelve a funcionar si se lleva el mando a 0 % y luego de nuevo a 100 % un tiempo después.

Causa (no es un error de software): es el **corte por baja tensión (LVC)** del ESC. Una batería 2S a 7 V está ya baja ($\approx 3,5$ V/celda); bajo la carga del motor la tensión cae por debajo del umbral de corte del ESC ($\approx 3,0$ – $3,2$ V/celda) y el ESC corta el motor para proteger el pack. Sin carga la tensión se recupera, el corte se rearma y el motor vuelve a girar un momento, produciendo justamente el comportamiento “a impulsos”. El efecto empeora mucho si el ESC tiene mal configurado el número de celdas.

Solución: encadenar en serie 2 baterías LiPo ($\approx 15,4$ V). Ningún cambio de código corrige un corte por baja tensión; el firmware envía una consigna estable.

12.3. Fallo al programar el barco — brownout durante el flash

Síntoma: la programación falla repetidamente (“*chip stopped responding*”, “*No serial data received*”) y el CP2104 se re-enumerar en mitad de la escritura. Esto solo pasa cuando la T-Beam está conectada directamente al PCB.

Causa: el mismo *brownout* de alimentación. El pico de corriente del borrado/escritura de flash desploma la tensión y el ESP32 se resetea.

Desconectar la T-beam del PCB, Programarla y sin desconectarla del USB volver a colocarla en el PCB.

12.4. Otros problemas de firmware ya resueltos

12.4.1. DUTY_MODE_1 en OPR_B — servo del timón inoperante

Doble inversión a través del BC548 (ver Sección 9.2). Solución: DUTY_MODE_0 en ambos canales.

12.4.2. Umbral de armado de ESC demasiado bajo

El mínimo real del manche de gas del PTR-6A es $\approx 1265 \mu\text{s}$, por lo que el umbral de armado de $1050 \mu\text{s}$ nunca se alcanzaba. Se subió a $1300 \mu\text{s}$. **Nota:** el gesto de armado se **eliminó** después, al pasar al modo manual unificado y al control bidireccional (era incompatible con el nuevo mapeo del gas).

12.4.3. Modo bloqueado en Automatic — confusión CH4/CH5

La versión inicial leía CH4 para el modo, cuando el conmutador está en CH5. Solución: leer `frame.ch5`.

12.4.4. Puerto serie ocupado al programar

Un monitor serie o script Python mantenía el puerto abierto (Errno 16: `Device or resource busy`). Solución: cerrar el monitor o `fuser /dev/ttyUSB0` y matar el proceso.

13. Problemas que encontramos — opciones de mejora

Esta sección reúne los problemas aún **abiertos** y, para cada uno, la opción de mejora propuesta para resolverlo. Son las líneas de trabajo prioritarias para que el dron pueda cumplir realmente su misión.

13.1. Alcance de LoRa muy pobre dentro de la caja de electrónica

Problema: con la antena LoRa actual ubicada **dentro de la caja de electrónica**, el alcance de radio es muy pobre. La caja actúa como pantalla y la proximidad de la electrónica degrada el patrón de radiación, reduciendo drásticamente el RSSI y el rango útil de telemetría y comandos.

Mejora propuesta: adquirir una **antena LoRa externa** (433 MHz) y ubicarla **sobre el mástil principal**, lo más alta y despejada posible, fuera de la caja. Esto mejora la línea de vista y aleja la antena de las masas metálicas y del ruido de la electrónica.

A considerar — conector: hay que verificar el conector de la antena y del módulo. El SX1276 del T-Beam suele exponer **u.FL/IPEX**, mientras que las antenas externas de 433 MHz traen habitualmente **SMA**. Será necesario el **cable adaptador / pigtail correspondiente (u.FL → SMA)** y cuidar que sea de 433 MHz (no de 2,4 GHz) para no degradar la adaptación.

13.2. Inconsistencias del GPS

Problema: el GPS presenta inconsistencias (fix intermitente, pocos satélites, posición poco estable), agravadas por la ubicación de la antena dentro o cerca de la caja y la electrónica.

Mejora propuesta: adquirir **otra antena externa, esta vez de GPS**, y ubicarla **sobre la cubierta**, con vista despejada del cielo y lejos de la antena LoRa

y de la electrónica.

A considerar — conector: igual que con LoRa, hay que verificar el conector. La entrada de antena GPS del NEO-6M es **u.FL**; las antenas GPS activas suelen venir con **SMA** o directamente con cable u.FL. Habrá que prever el **adaptador adecuado** y confirmar que la antena sea **activa** (el supervisor de antena ya está habilitado por software, ver Sección 7.2).

13.3. Sin medición de viento ni de la posición de la vela

Problema: el esquema actual **no mide de dónde viene el viento ni la posición real de la vela**. Como el winch del timón (Regatta ECO II) tampoco tiene realimentación de posición, **no es posible conocer la posición real del timón / safran** ni cerrar el lazo de control con datos reales: toda la navegación autónoma depende de estimaciones (viento deducido del track GPS, posición de timón supuesta a partir de la consigna).

Mejora propuesta: conseguir **2 sensores** que midan respectivamente la **dirección del viento** y la **posición de la vela**, y conectarlos al **bus I2C de la placa** a través del **conector ya previsto en el PCB** (GPI021/GPI022, que el firmware dejó libres al eliminar `Wire.end()`). Con la dirección del viento real y la posición real de la vela, sí sería posible deducir la posición efectiva del timón y cerrar correctamente el lazo de navegación.

Ventaja arquitectónica: el firmware ya reserva el bus I2C y el PCB ya expone el conector de expansión, por lo que la integración eléctrica es directa; faltaría el driver `SensorBus` en software.

13.4. Poco espacio para las baterías en la caja

Problema: la caja de electrónica tiene **poco espacio para las baterías**, lo que limita la capacidad embarcada y, por tanto, la autonomía y el margen de tensión bajo carga (relacionado directamente con el corte por baja tensión de la Sección 12.2).

Mejora propuesta: revisar el diseño mecánico de la caja para alojar un pack de mayor capacidad (o una mejor distribución de las celdas), teniendo en cuenta además el espacio que ocuparán los nuevos sensores y los conectores de las antenas externas. Conviene dimensionar el pack para mantener la tensión por encima del umbral LVC del ESC durante toda la misión.

14. Estado actual del sistema

Fase	Objetivo	Estado	Notas
1	Control manual RC	[OK] Completo	Modo unificado + motor bidireccional
2	GPS + navegación	[OK] Completo	Fix probado en exterior
3	LoRa + telemetría	[OK] Completo	TX/RX verificados
PCB V2	Cableado nuevo + I2C libre	[OK] Completo	Compilado y probado
5	Estimación de viento	[OK] Codificado	Sin validar en agua
6	Navegación autónoma	[OK] Codificado	Sin probar en agua, sin tests
4	Sensores I2C	[] Pendiente	Ver Sección 13
7	Registro / misión completa	[] Pendiente	SPIFFS, propulsión auto

14.1. Valores de calibración actuales (resumen)

Parámetro	Valor	Notas
SAIL_CENTER_US	1520 μ s	Centro Futaba (no 1500)
SAIL_PLUS/MINUS_US	1575 / 1465	$\pm 10^\circ$ — calibrar en banco
ROTOR_MIN/MAX_US	1417 / 1583	$\pm 90^\circ$ físico (manual)
ROTOR_AUTO_RANGE_DEG	20°	Recorrido usado en auto

Parámetro	Valor	Notas
ESC_REVERSE_MIN_US	1000 μ s	−100 % (atrás, bidireccional)
ESC_NEUTRAL_US	1500 μ s	Neutro / paro
ESC_MAX_US	2000 μ s	+100 % (adelante)
CH3_CENTER_US	1545 μ s	Centro del gas — verificar en banco
ESC_SLEW_US	30 μ s/tick	Limitación de variación
Razón divisor batería	5,683	(562k + 120k) / 120k